



CHAPTER 3

Terraform Configuration Files & Basics

Providers, resources, variables, and outputs — the building blocks of every Terraform project.

3.1 Structure of a Terraform Project

A typical project is made of a handful of purpose-built files.

terraform-project/

```
|— main.tf
|— variables.tf
|— outputs.tf
|— terraform.tfvars
|— providers.tf
|— terraform.tfstate
└— .terraform/
```

A clean layout keeps configuration, variables, and state clearly separated.

main.tf

Defines the infrastructure (resources)

variables.tf

Stores input variables

outputs.tf

Defines output values

terraform.tfvars

Stores variable values (optional)

providers.tf

Defines the provider configuration (optional)

terraform.tfstate

Stores the current state of the infrastructure

.terraform/

Directory containing provider plugins

3.2 Writing Basic Terraform Configuration

Every configuration specifies four essentials:



Provider

The cloud platform — AWS, Azure, GCP



Resources

Infrastructure components — EC2, S3, RDS



Variables

Dynamic input values



Outputs

Display important information

Example — creating an S3 bucket

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_s3_bucket" "my_bucket" {  
  bucket = "my-terraform-bucket-12345"  
}
```

Apply the configuration

```
$ terraform init  
$ terraform apply -auto-approve
```

→ This creates an S3 bucket in AWS.

3.3 Providers & Resources



Providers

A plugin that lets Terraform talk to an external service (AWS, Azure, GCP, Kubernetes, Vault, GitHub, and more).

```
provider "aws" {  
  region = "us-east-1"  
}
```

→ [View all providers on the Terraform Registry.](#)



Resources

Any infrastructure component Terraform manages — EC2, S3, RDS, and so on.

Example — EC2 instance

```
resource "aws_instance" "example" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
}
```



Data Sources

Fetch existing information from a provider instead of creating something new.

```
data "aws_ami" "latest_amazon_linux" {  
  most_recent = true  
  owners      = ["amazon"]  
  ...  
}
```

Usage: `ami = data.aws_ami.latest_amazon_linux.id`

3.4 Variables & Outputs

Parametrize configuration with variables, then surface results with outputs.

Variables

variables.tf

```
variable "bucket_name" {  
    description = "The name of the S3 bucket"  
    type        = string  
    default     = "my-default-bucket"  
}
```

main.tf

```
resource "aws_s3_bucket" "my_bucket" {  
    bucket = var.bucket_name  
}
```

```
$ terraform apply \  
-var="bucket_name=my-custom-bucket"
```

Outputs

outputs.tf

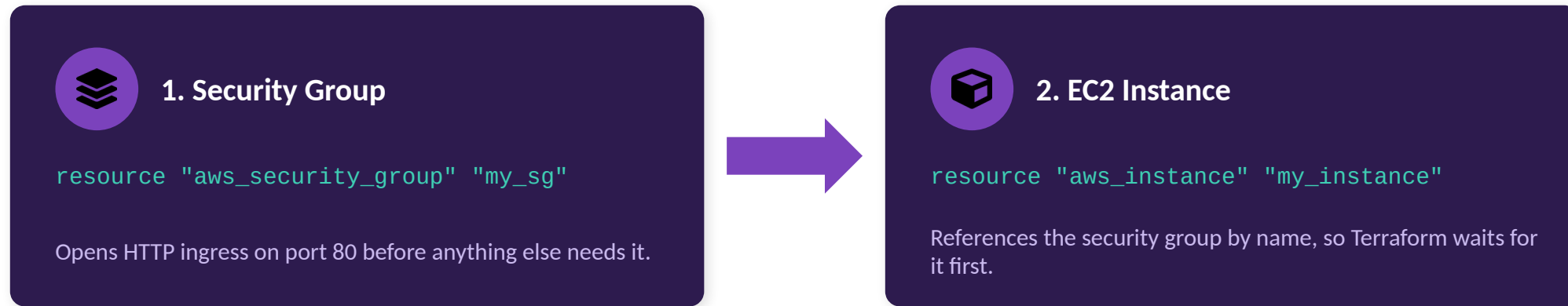
```
output "s3_bucket_name" {  
    description = "The name of the created bucket"  
    value       = aws_s3_bucket.my_bucket.id  
}
```

```
$ terraform output
```

→ Shows the bucket name directly in the terminal — no need to check the AWS console.

3.5 Dependencies in Terraform

Terraform automatically orders resource creation based on references between them.



→ Terraform builds a dependency graph and creates the security group before the instance — no manual ordering required.

```
resource "aws_security_group" "my_sg" { ... }  
  
resource "aws_instance" "my_instance" {  
  ...  
  security_groups = [aws_security_group.my_sg.name]  
}
```

Hands-On: Deploy an S3 Bucket

With variables and outputs, end to end.

1 Create the project

```
$ mkdir terraform-s3 &&  
cd terraform-s3
```

2 variables.tf

```
variable "bucket_name"  
{ type = string }  
variable "region"  
{ default = "us-east-1" }
```

3 main.tf

```
provider "aws" { region =  
var.region }  
resource "aws_s3_bucket"  
"my_bucket" {  
    bucket =  
var.bucket_name  
}
```

4 outputs.tf

```
output "bucket_id" {  
    value =  
aws_s3_bucket.my_bucket.i  
d  
}
```

Hands-On: Apply & Destroy

Bring the infrastructure up, confirm it, then tear it down.



5. Apply Terraform

```
$ terraform init
$ terraform apply \
  -var="bucket_name=my-terraform-bucket-123" \
  -auto-approve
```

→ Terraform creates the S3 bucket and prints the bucket name as an output.

Talking point: walk through the plan output line by line before typing yes.



6. Destroy the Infrastructure

```
$ terraform destroy -auto-approve
```

→ Deletes the S3 bucket to avoid ongoing AWS costs.

Always destroy demo resources at the end of a session — nothing should be left running in the account.

Summary

- ✓ Terraform project structure — main.tf, variables.tf, outputs.tf, and state
- ✓ Providers, resources, and data sources connect Terraform to the cloud
- ✓ Variables parametrize configuration; outputs surface key results
- ✓ Terraform resolves dependencies automatically via resource references
- ✓ Deployed an S3 bucket end-to-end using variables and outputs